

DEEP NEURAL NETWORKS (AIML ZG565) · COMPANION READER

# Activation Functions, Softmax, and Architecture Design

Four worked numerical problems: activations, softmax+CCE, vanishing gradients, and architecture sizing

Session 7 (7 June 2026) · Read alongside the lecture slides · Every slide example worked out in full

**How to use this companion.** Session 7 is a numerical examples session. Four distinct problems are worked through: (1) computing and comparing sigmoid, tanh, and ReLU outputs and derivatives on the same input vector; (2) softmax computation, categorical cross-entropy loss, and the gradient formula; (3) gradient flow analysis through a deep sigmoid network; (4) architecture sizing under a parameter budget. Colour code: **blue** intuition; **amber** everyday picture; **green** worked example; **red** watch out; **purple** key takeaway.

## 1 What this session covers

Session 7 continues the numerical examples begun in Session 6. The four problems build on each other conceptually: activation functions (sigmoid, tanh, ReLU) are the building blocks; softmax and cross-entropy are the output layer and loss for multi-class problems; gradient flow analysis shows concretely *why* sigmoid hidden layers cause vanishing gradients; and architecture design shows how to choose layer sizes in practice given data constraints.

Sessions 1–6 also left a complete forward + backprop problem (the same  $2 \rightarrow 3 \rightarrow 2$  network) which is revisited in slides 3–35 of this session as a revision exercise. The answers are unchanged from Session 6; refer to your Session 6 companion for the full working.

## 2 Problem 1: Activation functions on a vector input

**Problem (Slide 45).** Given input vector  $\mathbf{z} = [2.0, -1.5, 0.8, -0.3]^\top$ , compute: (1)  $\sigma(\mathbf{z})$  and  $\sigma'(\mathbf{z})$ ; (2)  $\tanh(\mathbf{z})$  and  $\tanh'(\mathbf{z})$ ; (3)  $\text{ReLU}(\mathbf{z})$  and  $\text{ReLU}'(\mathbf{z})$ .

### The intuition

All three are non-linear functions applied *element-wise*: each entry of  $\mathbf{z}$  is processed independently through the same scalar function. The outputs differ in range and how they handle negative inputs. The derivatives tell us how much gradient flows through each neuron during backpropagation.

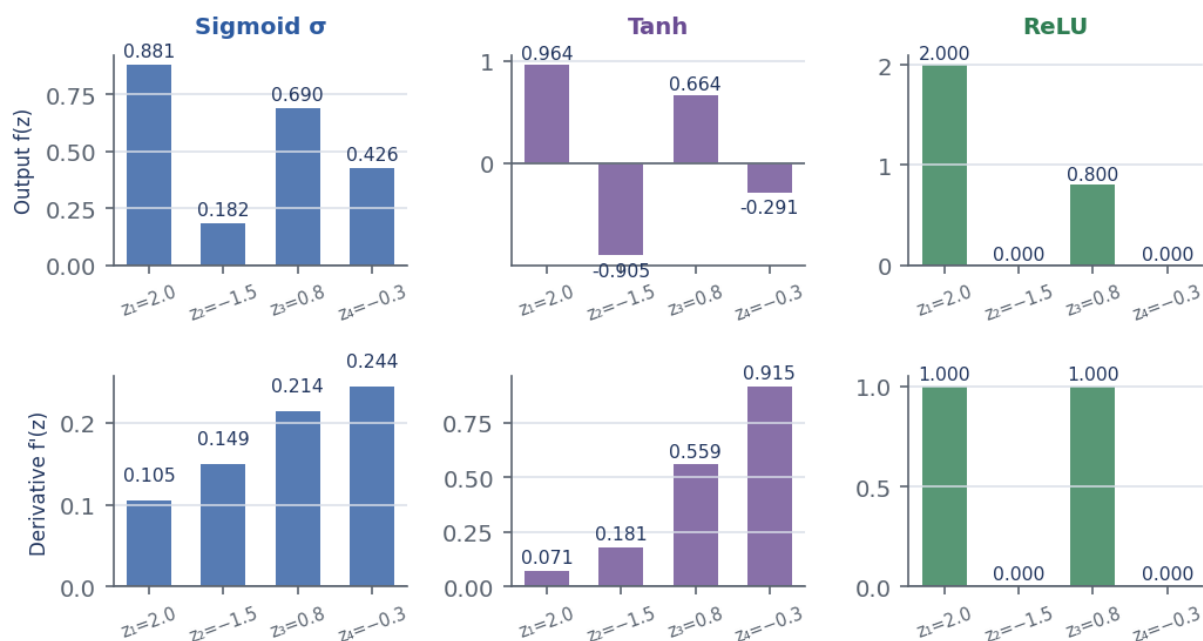
**Activation functions applied element-wise to  $\mathbf{z} = [2.0, -1.5, 0.8, -0.3]^T$** 

Figure 1: How to read it: top row — the output of each activation for each of the four  $z$  values. Bottom row — the derivative. Notice sigmoid and tanh derivatives shrink toward zero for large  $|z|$  (vanishing gradient), while ReLU derivative is 1 for  $z > 0$  and 0 for  $z \leq 0$  (binary mask).

**Worked example: Sigmoid  $\sigma(z) = 1/(1 + e^{-z})$  and derivative  $\sigma'(z) = \sigma(z)(1 - \sigma(z))$**

**Step 1. Compute each  $\sigma(z_i)$ :**

$$\begin{aligned}\sigma(2.0) &= 1/(1 + e^{-2.0}) = 1/(1 + 0.135) = \mathbf{0.881} \\ \sigma(-1.5) &= 1/(1 + e^{+1.5}) = 1/(1 + 4.482) = \mathbf{0.182} \\ \sigma(0.8) &= 1/(1 + e^{-0.8}) = 1/(1 + 0.449) = \mathbf{0.690} \\ \sigma(-0.3) &= 1/(1 + e^{+0.3}) = 1/(1 + 1.350) = \mathbf{0.426}.\end{aligned}$$

**Step 2. Compute derivatives using  $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ :**

$$\begin{aligned}\sigma'(2.0) &= 0.881(1 - 0.881) = 0.881 \times 0.119 = \mathbf{0.105} \\ \sigma'(-1.5) &= 0.182(1 - 0.182) = 0.182 \times 0.818 = \mathbf{0.149} \\ \sigma'(0.8) &= 0.690(1 - 0.690) = 0.690 \times 0.310 = \mathbf{0.214} \\ \sigma'(-0.3) &= 0.426(1 - 0.426) = 0.426 \times 0.574 = \mathbf{0.245}.\end{aligned}$$

**Step 3. Sense-check.** All outputs in  $(0, 1)$ . All derivatives positive and  $\leq 0.25$  (the maximum sigmoid derivative, attained at  $z = 0$ ). The large  $|z|$  values (2.0, -1.5) give small derivatives (0.105, 0.149) — gradients are already being squeezed.

 **Worked example: Tanh**  $\tanh(z) = (e^z - e^{-z}) / (e^z + e^{-z})$  and derivative  $\tanh'(z) = 1 - \tanh^2(z)$

**Step 1. Compute each  $\tanh(z_i)$ :**

$$\begin{aligned}\tanh(2.0) &= (7.389 - 0.135) / (7.389 + 0.135) = 7.254 / 7.524 = \mathbf{0.964} \\ \tanh(-1.5) &= (0.223 - 4.482) / (0.223 + 4.482) = -4.259 / 4.705 = \mathbf{-0.905} \\ \tanh(0.8) &= (2.226 - 0.449) / (2.226 + 0.449) = 1.777 / 2.675 = \mathbf{0.664} \\ \tanh(-0.3) &= (0.741 - 1.350) / (0.741 + 1.350) = -0.609 / 2.091 = \mathbf{-0.291}\end{aligned}$$

**Step 2. Derivatives  $\tanh'(z) = 1 - \tanh^2(z)$ :**

$$\begin{aligned}\tanh'(2.0) &= 1 - 0.964^2 = 1 - 0.929 = \mathbf{0.070} \\ \tanh'(-1.5) &= 1 - (-0.905)^2 = 1 - 0.819 = \mathbf{0.181} \\ \tanh'(0.8) &= 1 - 0.664^2 = 1 - 0.441 = \mathbf{0.559} \\ \tanh'(-0.3) &= 1 - (-0.291)^2 = 1 - 0.085 = \mathbf{0.915}\end{aligned}$$

**Step 3. Sense-check.** Outputs in  $(-1, 1)$ ; zero-centred (unlike sigmoid). Derivatives up to 1.0 (stronger than sigmoid's max 0.25). Still saturates at large  $|z|$ : derivative at  $z = 2.0$  is only 0.070.

 **Worked example: ReLU**  $\text{ReLU}(z) = \max(0, z)$  and derivative  $\text{ReLU}'(z) = 1[z > 0]$

**Step 1. Apply ReLU element-wise:**

$$\begin{aligned}\text{ReLU}(2.0) &= \max(0, 2.0) = \mathbf{2.0} \\ \text{ReLU}(-1.5) &= \max(0, -1.5) = \mathbf{0.0} \quad \leftarrow \text{dead} \\ \text{ReLU}(0.8) &= \max(0, 0.8) = \mathbf{0.8} \\ \text{ReLU}(-0.3) &= \max(0, -0.3) = \mathbf{0.0} \quad \leftarrow \text{dead}\end{aligned}$$

**Step 2. Derivatives (binary mask):**

$$\begin{aligned}\text{ReLU}'(2.0) &= \mathbf{1} \quad (z > 0). \\ \text{ReLU}'(-1.5) &= \mathbf{0} \quad (z \leq 0). \\ \text{ReLU}'(0.8) &= \mathbf{1} \quad (z > 0). \\ \text{ReLU}'(-0.3) &= \mathbf{0} \quad (z \leq 0).\end{aligned}$$

**Step 3. Sense-check.** Output = input when positive, 0 otherwise. Derivative mask: active neurons pass gradient unchanged (1); dead neurons block it completely (0). No vanishing for active neurons.

#### ✓ Key takeaway

Key comparison for 4 inputs:  $\sigma$ : outputs (0.881, 0.182, 0.690, 0.426), derivatives (0.105, 0.149, 0.214, 0.245). Tanh: outputs (0.964, -0.905, 0.664, -0.291), derivatives (0.070, 0.181, 0.559, 0.915). ReLU: outputs (2.0, 0.0, 0.8, 0.0), derivatives (1, 0, 1, 0).

## 3 Problem 2: Softmax, categorical cross-entropy, and gradient

**Problem (Slide 49).** Given logit vector  $\mathbf{z} = [2.0, 1.0, 0.1, -1.0]^\top$  and true one-hot label  $\mathbf{y} = [0, 1, 0, 0]^\top$  (class 2 is correct), compute: (1) softmax probabilities  $\hat{\mathbf{y}}$ ; (2) cross-entropy loss  $L$ ; (3) gradient  $\partial L / \partial \mathbf{z}$ .

 **Worked example: Softmax computation with numerical stability trick**

**Step 1. Subtract max for stability.**  $\max(\mathbf{z}) = 2.0$ :

$$\mathbf{z}_{\text{stable}} = \mathbf{z} - 2.0 = [0.0, -1.0, -1.9, -3.0]^\top. \quad (1)$$

Why? Exponentiating large positive numbers risks overflow. Subtracting the max makes all values  $\leq 0$  (exponentials  $\leq 1$ ), giving the same softmax output (the constant cancels in numerator and denominator).

**Step 2. Exponentiate.**

$$e^{\mathbf{z}_{\text{stable}}} = [e^0, e^{-1}, e^{-1.9}, e^{-3}] = [1.000, 0.368, 0.150, 0.050]. \quad (2)$$

**Step 3. Sum.**  $S = 1.000 + 0.368 + 0.150 + 0.050 = 1.568$ .

**Step 4. Divide.**

$$\hat{y}_1 = 1.000 / 1.568 = \mathbf{0.638}, \quad (3)$$

$$\hat{y}_2 = 0.368 / 1.568 = \mathbf{0.235}, \quad (4)$$

$$\hat{y}_3 = 0.150 / 1.568 = \mathbf{0.096}, \quad (5)$$

$$\hat{y}_4 = 0.050 / 1.568 = \mathbf{0.032}. \quad (6)$$

**Step 5. Verify.**  $0.638 + 0.235 + 0.096 + 0.032 = 1.001 \approx 1$  (rounding). ✓

 **Worked example: Categorical cross-entropy loss**

Since  $\mathbf{y} = [0, 1, 0, 0]^\top$  is one-hot (class 2 is correct), only the second term contributes:

$$L = - \sum_{i=1}^4 y_i \log(\hat{y}_i) = -y_2 \log(\hat{y}_2) = -1 \times \log(0.235) = \mathbf{1.447}. \quad (7)$$

Sense-check: if the model assigned all probability to class 2,  $L = 0$ . A random model gives  $L = \log(4) \approx 1.386$ . Our  $L = 1.447$  is slightly worse than random — the model is actually spreading probability *away* from class 2 (it assigns 63.8% to class 1 which is wrong). There is a lot of room to improve.

 **Worked example: Gradient of cross-entropy with softmax**

The elegant formula  $\partial L / \partial \mathbf{z} = \hat{\mathbf{y}} - \mathbf{y}$ :

$$\frac{\partial L}{\partial \mathbf{z}} = \begin{bmatrix} 0.638 \\ 0.235 \\ 0.096 \\ 0.032 \end{bmatrix} - \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} \mathbf{0.638} \\ \mathbf{-0.765} \\ \mathbf{0.096} \\ \mathbf{0.032} \end{bmatrix}. \quad (8)$$

Interpretation: class 1 has gradient +0.638 (model over-predicted it; push down); class 2 has gradient -0.765 (under-predicted; the correct class needs to go up); classes 3 and 4 have small positive gradients (also over-predicted but slightly).

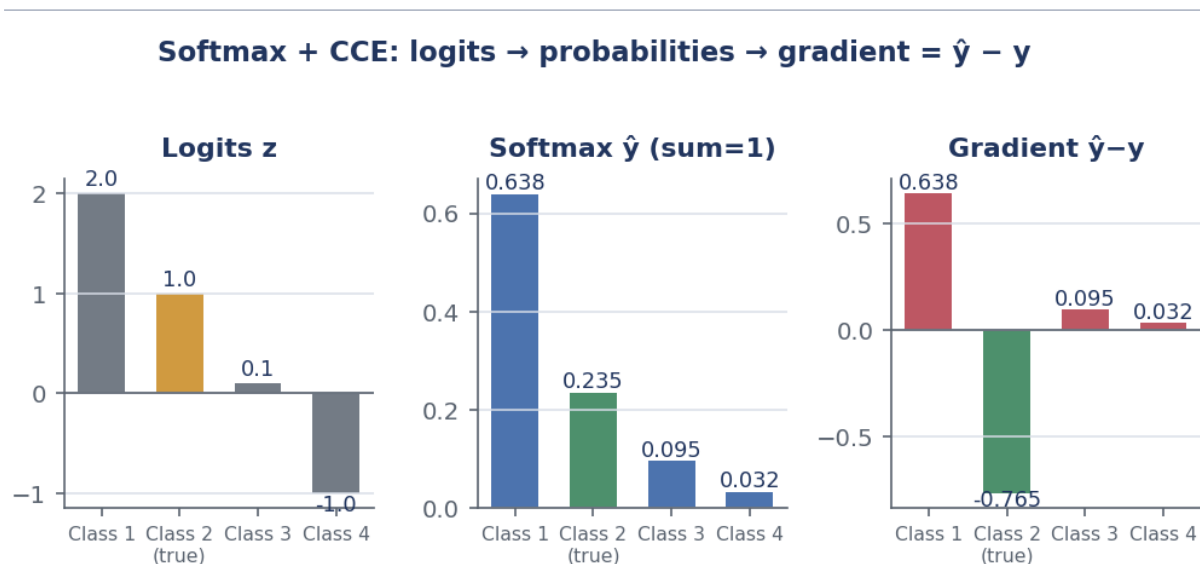


Figure 2: How to read it: left — raw logits (class 1 gets the highest logit 2.0). Middle — after softmax, they become a probability distribution; class 1 gets 63.8% but should get 0%. Right — the gradient  $\hat{y} - y$ : class 2 (correct, in green) has a negative gradient (needs increasing); others are positive (need decreasing).

✓ **Key takeaway**

Softmax + CCE gradient:  $\partial L / \partial \mathbf{z} = \hat{\mathbf{y}} - \mathbf{y}$ . For the correct class, the gradient is negative (push the logit up). For wrong classes, it is positive (push them down). The magnitude tells us how much the model was off.

## 4 Problem 3: Gradient flow analysis in a deep sigmoid network

**Problem (Slide 54).** A 4-layer network with sigmoid activations. All weight matrices have spectral norm  $\|W^{[\ell]}\| = 0.8$ . Sigmoid derivatives at each layer:  $g'^{[\ell]} = 0.25$  (maximum). Initial gradient magnitude at the output:  $|\delta^{[4]}| = 1.0$ .

**Formula.** The gradient magnitude decays as it propagates backward:

$$|\delta^{[\ell-1]}| = |\delta^{[\ell]}| \cdot \|W^{[\ell]}\| \cdot |g'^{[\ell-1]}|. \quad (9)$$

**Worked example: Layer-by-layer gradient decay**

**Step 1. Layer 4 (output):**  $|\delta^{[4]}| = 1.0$  (given).

**Step 2. Layer 3:**  $|\delta^{[3]}| = 1.0 \times 0.8 \times 0.25 = 0.200$ .

**Step 3. Layer 2:**  $|\delta^{[2]}| = 0.2 \times 0.8 \times 0.25 = 0.040$ .

**Step 4. Layer 1:**  $|\delta^{[1]}| = 0.04 \times 0.8 \times 0.25 = 0.008$ .

**Step 5. What happened?** Going from layer 4 to layer 1, the gradient shrank by a factor of  $0.8 \times 0.25 = 0.2$  at *every step*. After 3 backward steps, the gradient is  $0.2^3 = 0.008$  of its original magnitude — less than 1% of the output gradient reaches the first layer.

**Step 6. Implications.** The first layer receives almost no learning signal. Its weights barely update, so it cannot learn useful low-level features. Deeper networks with sigmoid activations suffer increasingly from this.

### Vanishing gradient problem: sigmoid kills gradients in deep networks

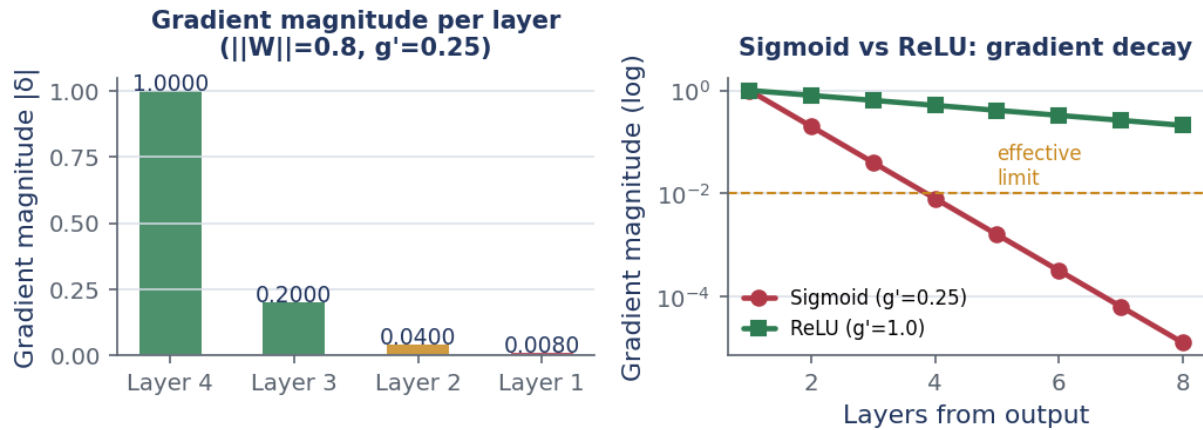


Figure 3: How to read it: left — the gradient magnitude collapses from 1.0 at the output to 0.008 at layer 1. Right — with ReLU ( $g' = 1$  for active neurons), the decay is only from  $\|W\| = 0.8$  per layer, and for 8 layers it remains  $0.8^8 \approx 0.17$  vs sigmoid's  $0.2^8 \approx 0.00006$  — a factor of  $2800\times$  more gradient reaching layer 1.

#### ✓ Key takeaway

Vanishing gradient: per-step decay factor =  $\|W\| \times g'_{\max}$ . For sigmoid:  $0.8 \times 0.25 = 0.2$ . After 3 steps:  $0.2^3 = 0.008$ . Fix: ReLU ( $g' = 1$  for active neurons gives factor 0.8 not 0.2).

## 5 Problem 4: Architecture design under data constraints

When designing a DFNN, the number of layers  $L$  and neurons per layer  $n_\ell$  must be chosen so that the model does not catastrophically overfit. A commonly cited heuristic:

$$\text{parameters} \leq M/5, \quad (10)$$

where  $M$  is the number of training examples. (Some sources use  $M/10$ .) Additionally, the maximum useful depth is bounded by:

$$L \leq \log_2 \left( \frac{M}{d \times c} \right) + 1, \quad (11)$$

where  $d$  is the input dimension and  $c$  is the number of output classes.

### 5.1 Example 1: Small tabular dataset

$d = 20$  features (customer data),  $M = 800$  training samples,  $c = 3$  classes. Parameter budget:  $M/5 = 160$ .

#### ✍ Worked example: Finding the architecture for Example 1

**Step 1. Max depth.**  $L \leq \log_2(800/(20 \times 3)) + 1 = \log_2(13.33) + 1 \approx 3.74 + 1 \approx 4.7$ . Use  $L \leq 4$ .

**Step 2. Try  $L = 2$ , start wide.** Architecture  $[20, 40, 3]$ : params =  $20 \times 40 + 40 + 40 \times 3 + 3 = 840 + 163 = 963$ . Way over budget ( $963 \gg 160$ ).

**Step 3. Reduce width.**  $[20, 20, 3]$ : params =  $420 + 23 = 443$ . Still too many.

**Step 4. Reduce more.**  $[20, 10, 3]$ :  $\text{params} = 210 + 33 = 243$ . Still over.

**Step 5. Try 6 units.**  $[20, 6, 3]$ :  $\text{params} = 20 \times 6 + 6 + 6 \times 3 + 3 = 126 + 21 = 147$ .  $147 \leq 160$ . ✓

**Step 6. Final recommendation:** architecture  $[20, 6, 3]$  with regularisation (dropout or L2, since 147 parameters on 800 examples is still moderately complex).

## 5.2 Example 2: Medium image dataset

$d = 784$  features (28×28 grayscale images = MNIST-like),  $M = 5,000$  training samples,  $c = 10$  classes (digit classification). Parameter budget:  $M/5 = 1,000$ .

### Worked example: Finding the architecture for Example 2

**Step 1. Max depth.**  $L \leq \log_2(5000/(784 \times 10)) + 1 = \log_2(0.638) + 1 \approx -0.65 + 1 = 0.35$ . This is less than 1, meaning even  $L = 1$  (one hidden layer) is at the ragged edge for this heuristic.

**Step 2. The dimension problem.** A single hidden layer  $[784, n, 10]$  has  $784n + n + 10n + 10 = (794)n + 10$  parameters. For budget 1,000:  $n \approx 1$ . This is far too small to learn digit features.

**Step 3. Resolution.** In practice, for structured image data, the heuristic is applied more loosely. A common starting point is  $[784, 128, 64, 10]$ :  $\text{params} = 784 \times 128 + 128 + 128 \times 64 + 64 + 64 \times 10 + 10 = 100,480 + 8,256 + 650 = 109,386$ .

**Step 4. The real lesson.** For image data, DFNN with fully-connected layers struggle: even a modest architecture has too many parameters. This motivates Convolutional Neural Networks (CNNs), which use parameter sharing (one filter applied across the whole image) to reduce parameter count by orders of magnitude.

### Architecture design: balancing capacity vs parameter budget

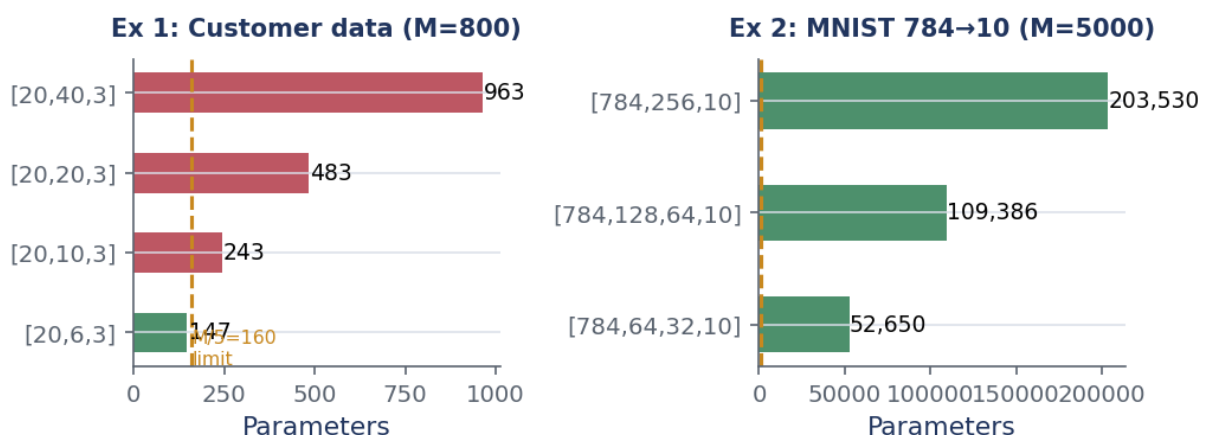


Figure 4: How to read it: left — for the small customer dataset, reducing hidden layer width from 40 to 6 brings parameters within the budget of 160. Right — for MNIST-like images, even modest DFNN architectures have far more parameters than the simple heuristic allows; CNNs (parameter sharing) are the solution.

**✓ Key takeaway**

Architecture guideline: parameters  $\leq M/5$ ; depth  $\leq \log_2(M/(d \cdot c)) + 1$ . For small tabular data, start narrow ( $n \approx 6$ –20). For image data, prefer CNNs over fully-connected layers to manage parameter counts.

## 6 Recap and what comes next

One paragraph per problem:

**Activation functions.** Sigmoid: outputs (0.881, 0.182, 0.690, 0.426); derivatives (0.105, 0.149, 0.214, 0.245) – all  $\leq 0.25$ . Tanh: outputs (0.964,  $-0.905$ , 0.664,  $-0.291$ ); derivatives (0.070, 0.181, 0.559, 0.915). ReLU: outputs (2.0, 0.0, 0.8, 0.0); derivatives (1, 0, 1, 0).

**Softmax + CCE.**  $\mathbf{z} = [2, 1, 0.1, -1]$ : softmax gives  $\hat{\mathbf{y}} = [0.638, 0.235, 0.096, 0.032]$ ; loss  $L = 1.447$ ; gradient  $\partial L / \partial \mathbf{z} = [0.638, -0.765, 0.096, 0.032]$ .

**Gradient flow.** Per-step decay =  $\|W\| \cdot g'_{\max} = 0.8 \times 0.25 = 0.2$ . Three steps back:  $0.2^3 = 0.008$  (less than 1% reaches the first layer).

**Architecture.** Small tabular: [20, 6, 3] (147 params  $\leq$  160 budget). Image data: fully-connected is parameter-inefficient; CNNs are the fix.

**✓ Key takeaway**

Use ReLU to avoid vanishing gradients; use softmax+CCE for multi-class output; size architectures with the parameter budget rule; prefer CNNs over fully-connected layers for image data. Session 8 covers CNNs.

**Further reading.** Zhang et al. *Dive into Deep Learning*, Chapters 4–6; Goodfellow et al., Chapter 6; LeCun et al. (1998) for the original CNN paper.